



Applied Generative AI & LLM Systems

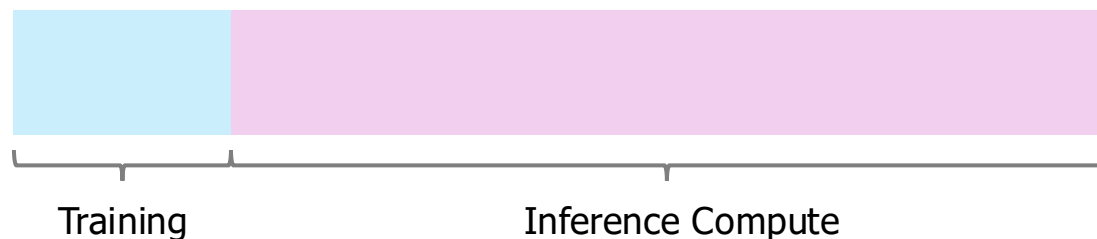
Theory behind efficient LLM inference

2026.03.30

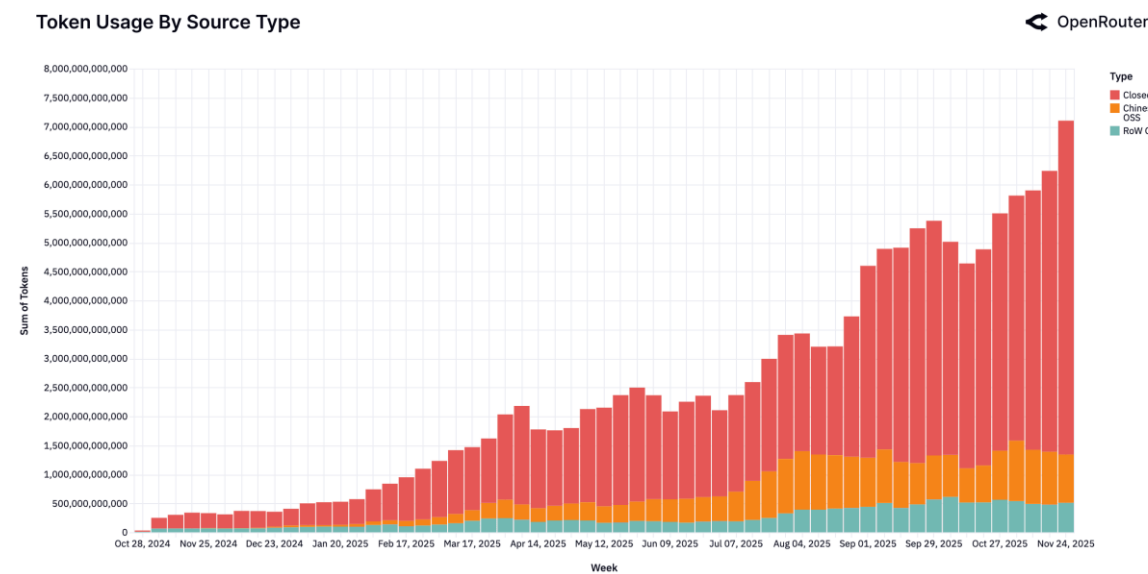
Johannes Oster • johannes.oster@tuwien.ac.at

Motivation

Overall Lifetime Cost of LLMs



Overall Token Usage Increases



<https://openrouter.ai/state-of-ai>

Loads of Terminology Floating Around

TheBloke/CapybaraHermes-2.5-Mistral-7B-GPTQ

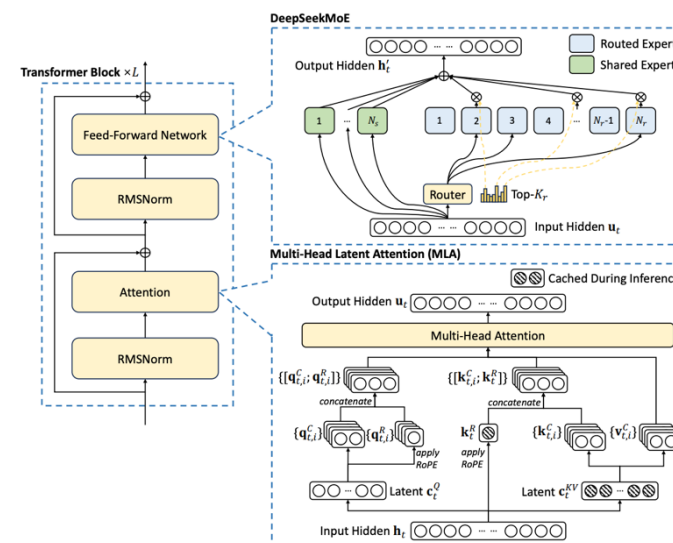
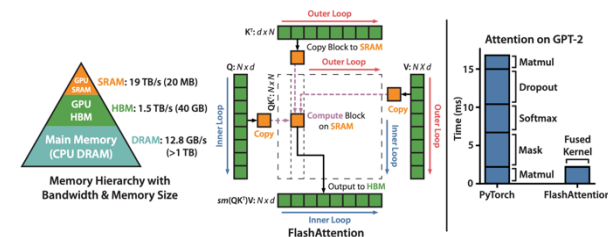
Updated Jan 31, 2024 • 335 • 61

TheBloke/CapybaraHermes-2.5-Mistral-7B-AWQ

Updated Jan 31, 2024 • 24 • 23

TheBloke/CapybaraHermes-2.5-Mistral-7B-GGUF

7B • Updated Jan 31, 2024 • 2.05k • 126



AGENDA

1 GPU primer

2 Recap: Inference with Transformer-based models

3 Overview of techniques for efficient LLM inference

4 Recap / Closing Remarks



Compute, Memory, Memory Bandwidth
GPU Primer

Compute

The **computational capacity** of a processor measured in **F**loating **P**oint **O**perations per **S**econd (FLOPS).



Memory Capacity*

The **data storage capacity** of a processor measured in Bytes.



Memory Bandwidth

The **rate** at which data can be **written** to/**read** from **memory** measured in Bytes per Second (B/s).



H100

989.5 TFLOPS/s
in FP16

80GB

3.35TB/s
(3.35 GB/ms)

Roughly 300 operations in the time it takes to read / write 1 byte

*sometimes referred to as only "memory"

focus of this lecture

Compute

The **computational capacity** of a processor measured in **Floating Point Operations per Second** (FLOPS).



Memory Capacity*

The **data storage capacity** of a processor measured in Bytes.



Memory Bandwidth

The **rate** at which data can be **written to/read** from **memory** measured in Bytes per Second (B/s).



H100

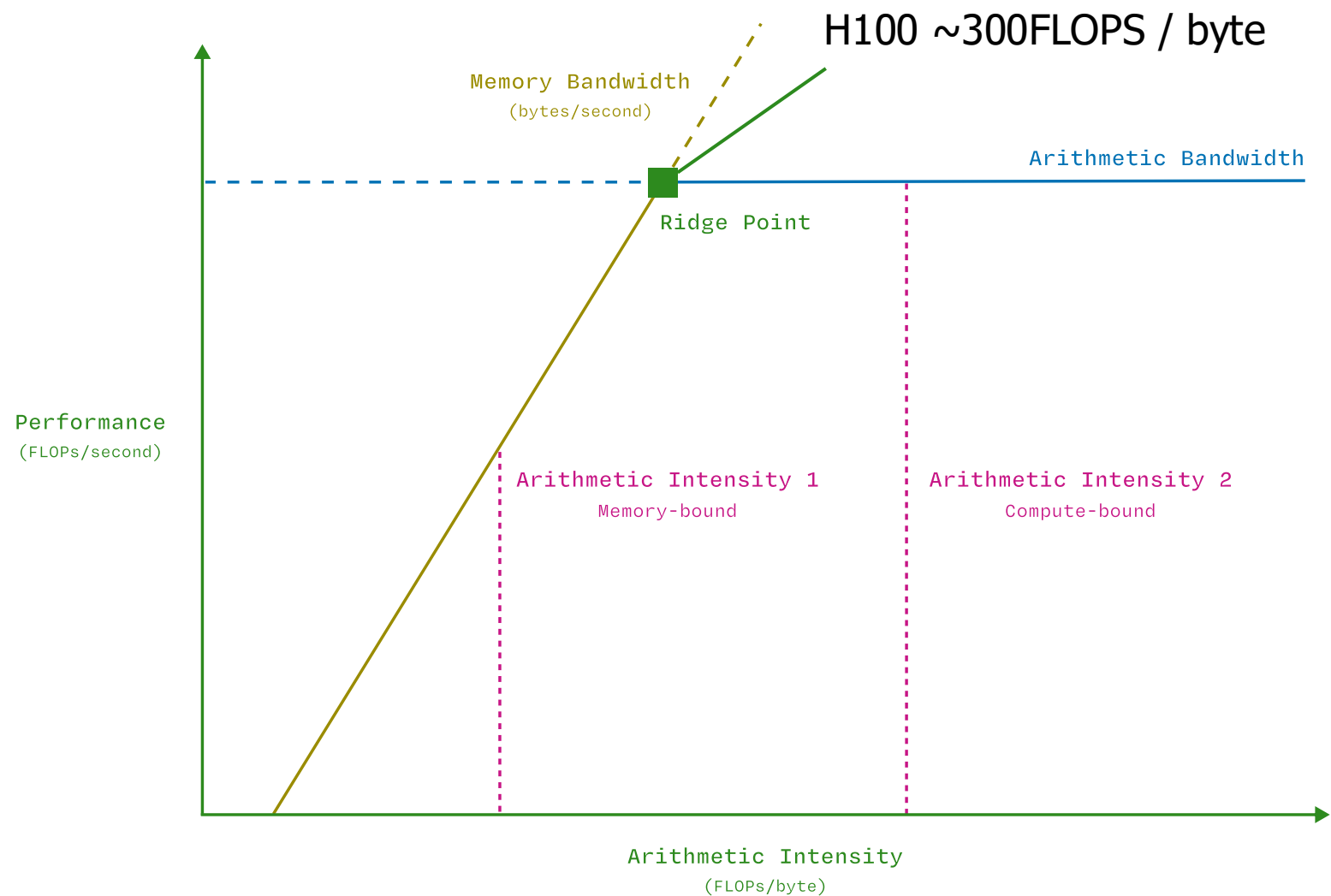
989.5 TFLOPS/s
in FP16

80GB

3.35TB/s
(3.35 GB/ms)

Roughly 300 operations in the time it takes to read / write 1 byte

*sometimes referred to as only "memory"



Arithmetic Bandwidth
= Compute

Arithmetic Intensity
Ratio between FLOPS to memory transfer.

$$\frac{FLOPS}{Memory\ Access}$$

High Level Overview of Efficient LLM Inference

Data Level

The **model** and **serving stack** stay **unchanged**, but **less data** enters the model / flows through the network.

E.g. Prompt Compression, Token Merging, Token Pruning

Model Level

Operates on the **model** itself (i.e. its weights, attention math, decoding logic, and architecture). The **input** and **serving stack** stay **unchanged**.

E.g. Quantization, Attention-Optimizations, Conditional Weight Activations, ...

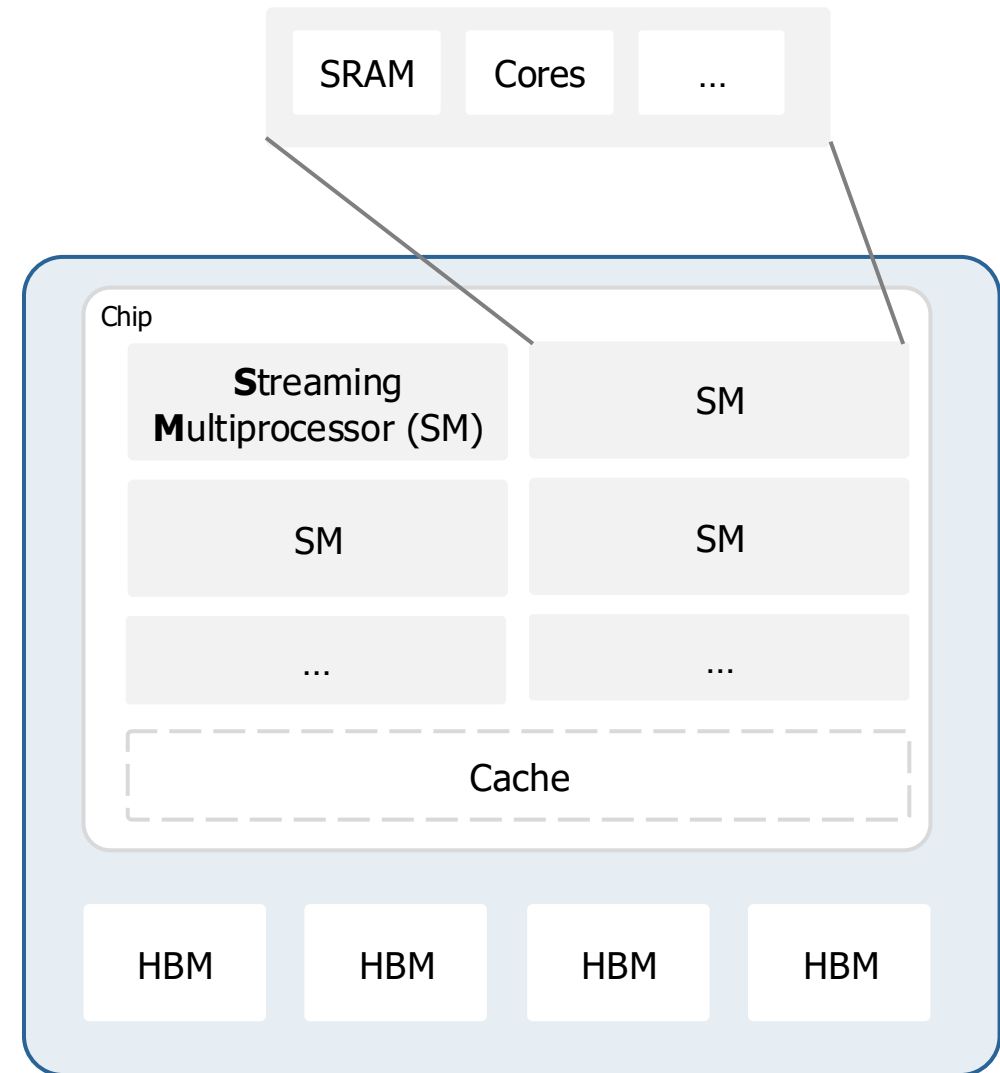
System Level

Operates on the **serving stack** around the model (i.e. how, where, and when computation runs). The **model** and **input** stay unchanged.

E.g. KV cache management, alternative hardware designs, continuous batching, ...

Disclaimer: We will not cover this in this lecture

- **VRAM (Video RAM)**
Any off-chip memory dedicated to the GPU. **H**igh **B**andwidth **M**emory (HBM) is an example of VRAM usually found on cloud GPUs. e.g. HBM 80 GB capacity with 3 TB/s bandwidth
- **Streaming Multiprocessor**
One of many parallel processors on the chip (e.g. 132 on H100). Each SM has its own SRAM and cores.
- **SRAM**
Very fast on-chip memory. e.g. SRAM 228 KB capacity with 30 TB/s bandwidth (per SM)
- **Cache**
Hardware-managed memory between SMs and HBM. Recently-accessed HBM data lives here. e.g. 50MB capacity with 5 TB/s bandwidth
- **Kernel**
A function launched from the CPU that runs on the GPU. Reads inputs from HBM, computes via SMs, writes outputs back.



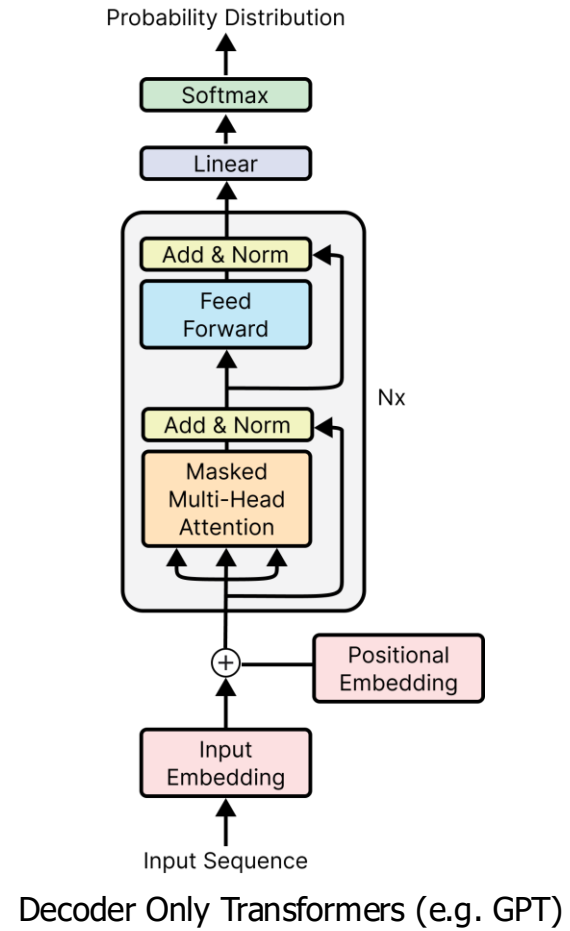
GPU Module

2

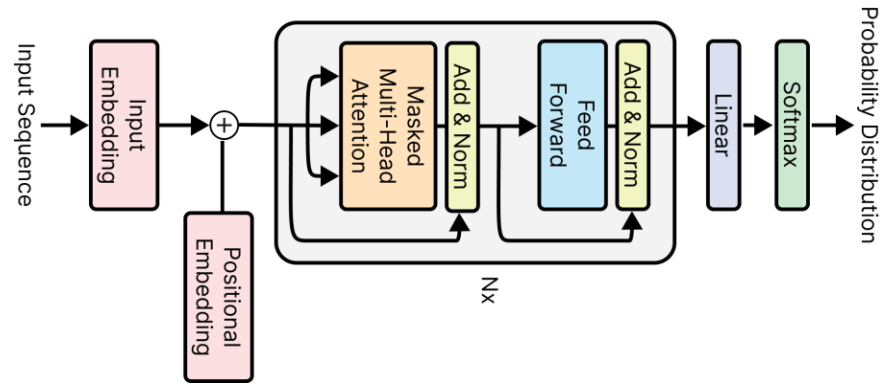
Recap

Transformer Architecture

Recap – Transformer Architecture



Recap – Transformer Architecture



We like dragons

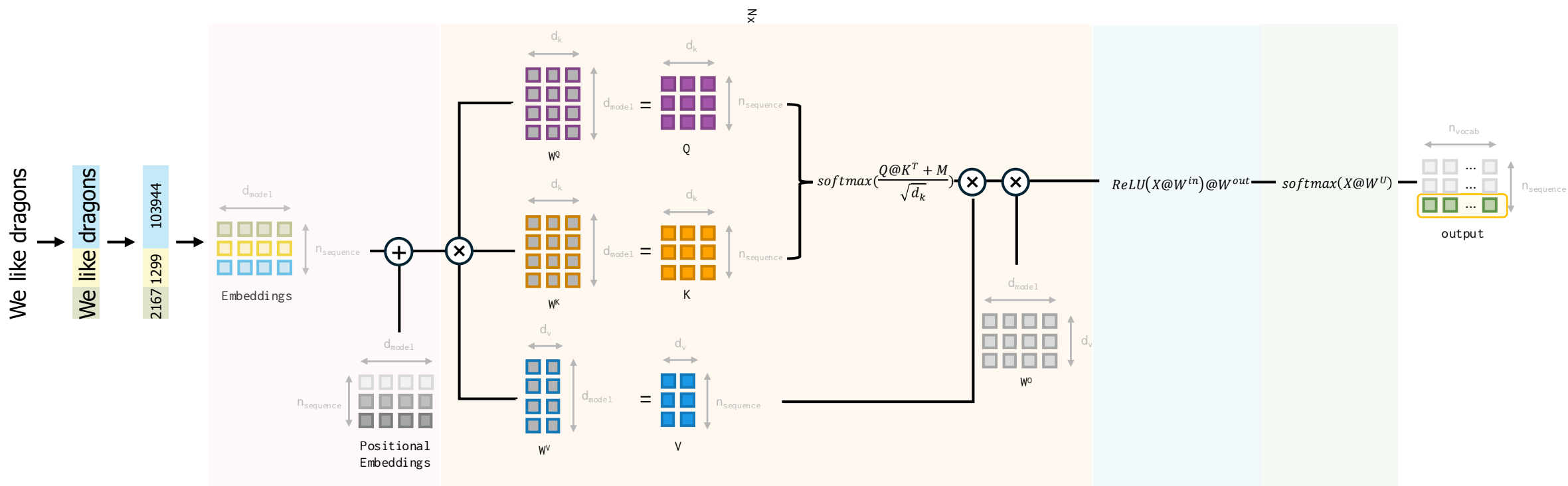
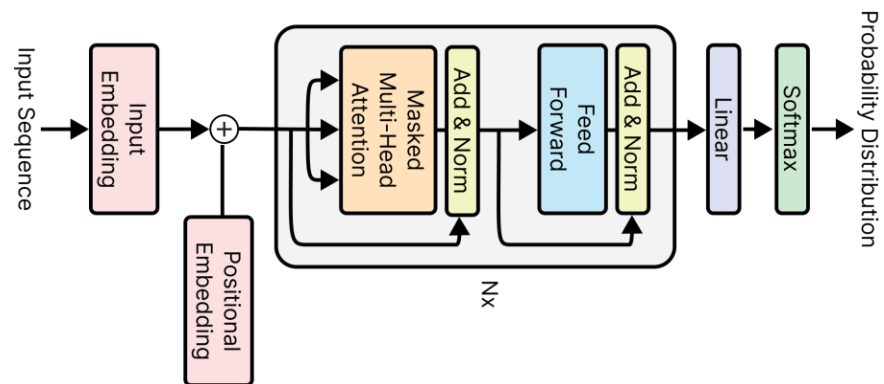


We like dragons



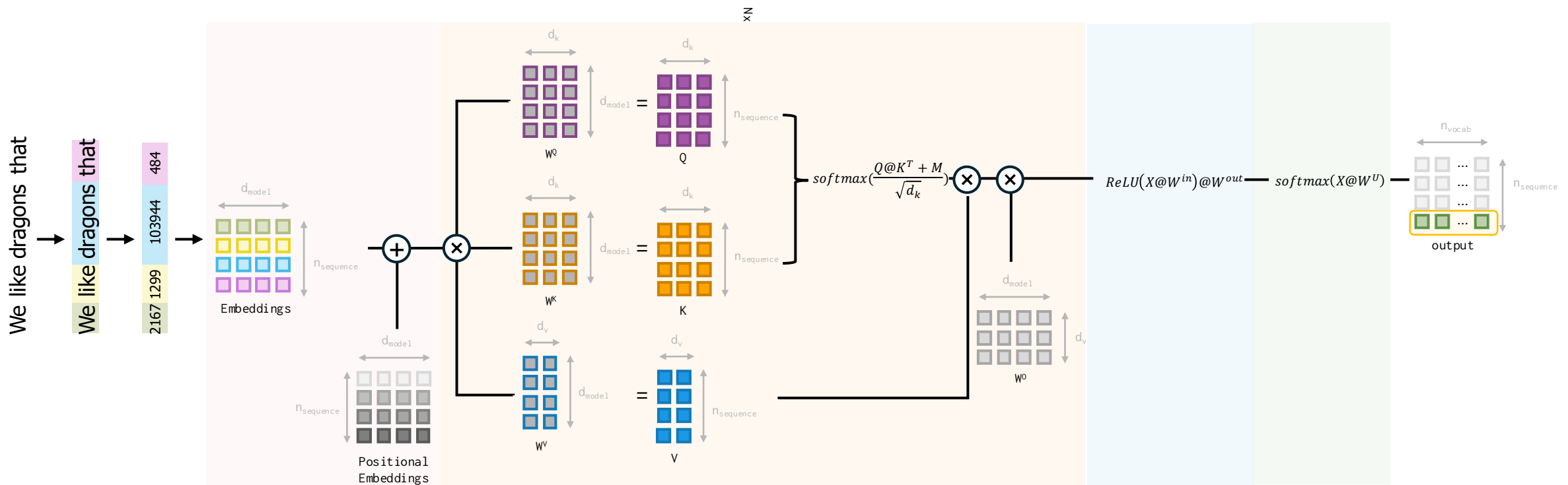
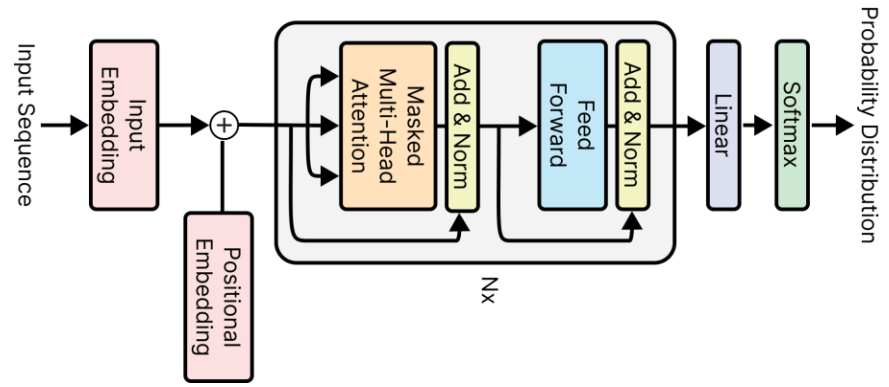
2167 1299 103944

Recap – Transformer Architecture| Prefilling



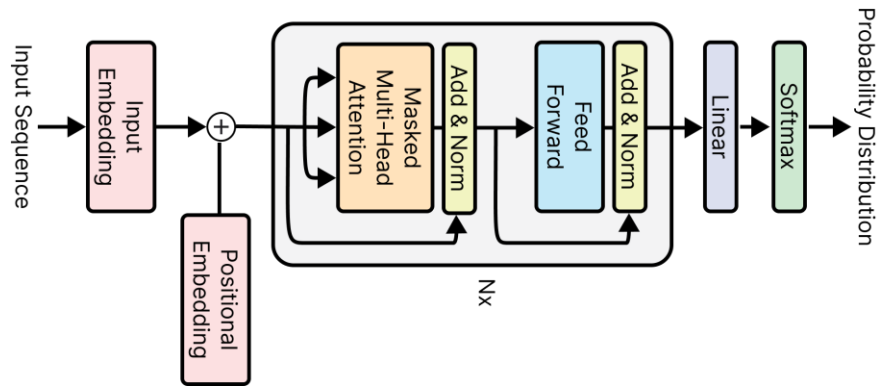
Layer norms, residual connections and head multiplicity are hidden for simplicity

Recap – Transformer Architecture | Decoding



Layer norms, residual connections and head multiplicity are hidden for simplicity

Recap – Transformer Architecture | Decoding



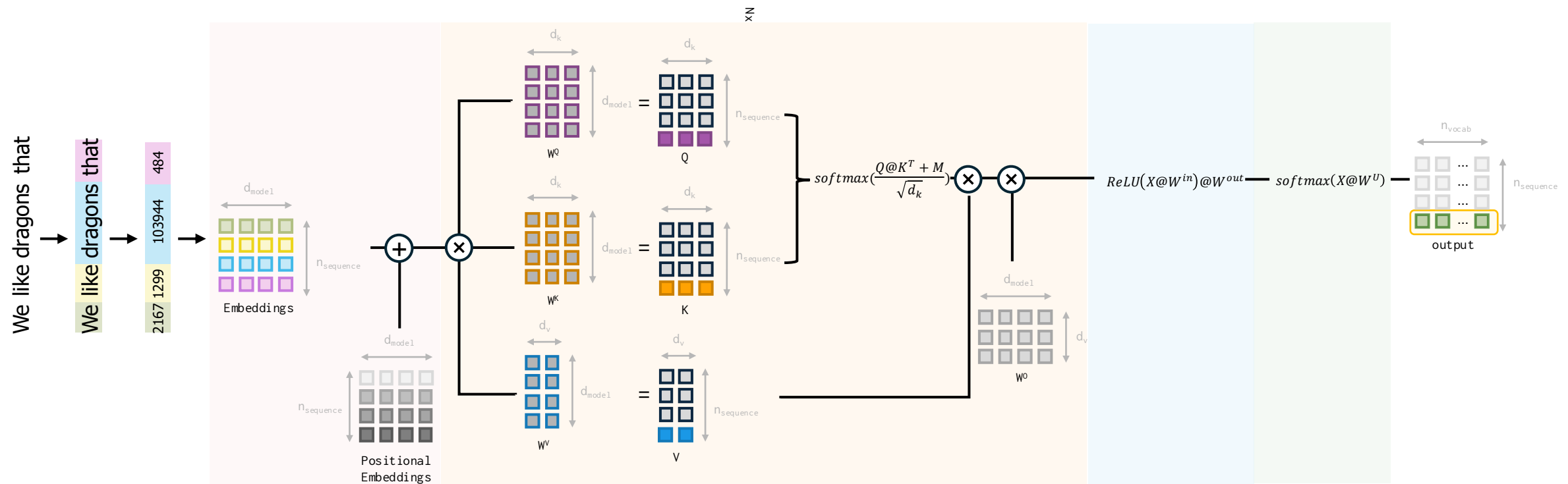
□ □ □ □

i Computed values of past tokens don't change during inference (causal nature of LLMs)

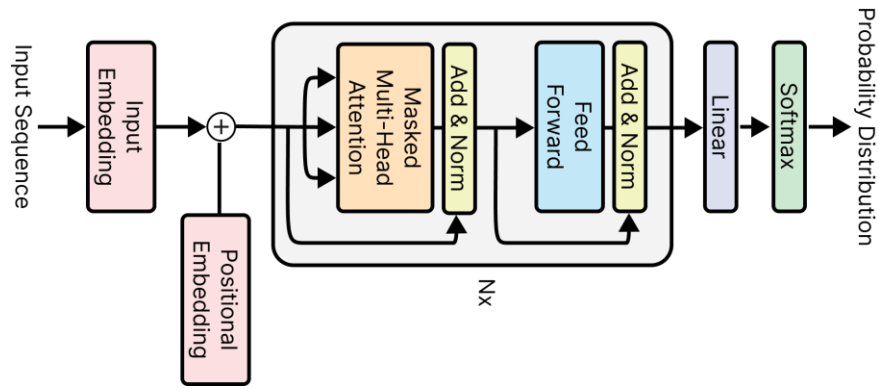
→ Cache

□ □ ... □

i Past tokens are only necessary in the attention mechanism. Their probability distribution are discarded.



Recap – Transformer Architecture | Decoding



□ □ □ □

i Computed values of past tokens don't change during inference (causal nature of LLMs)

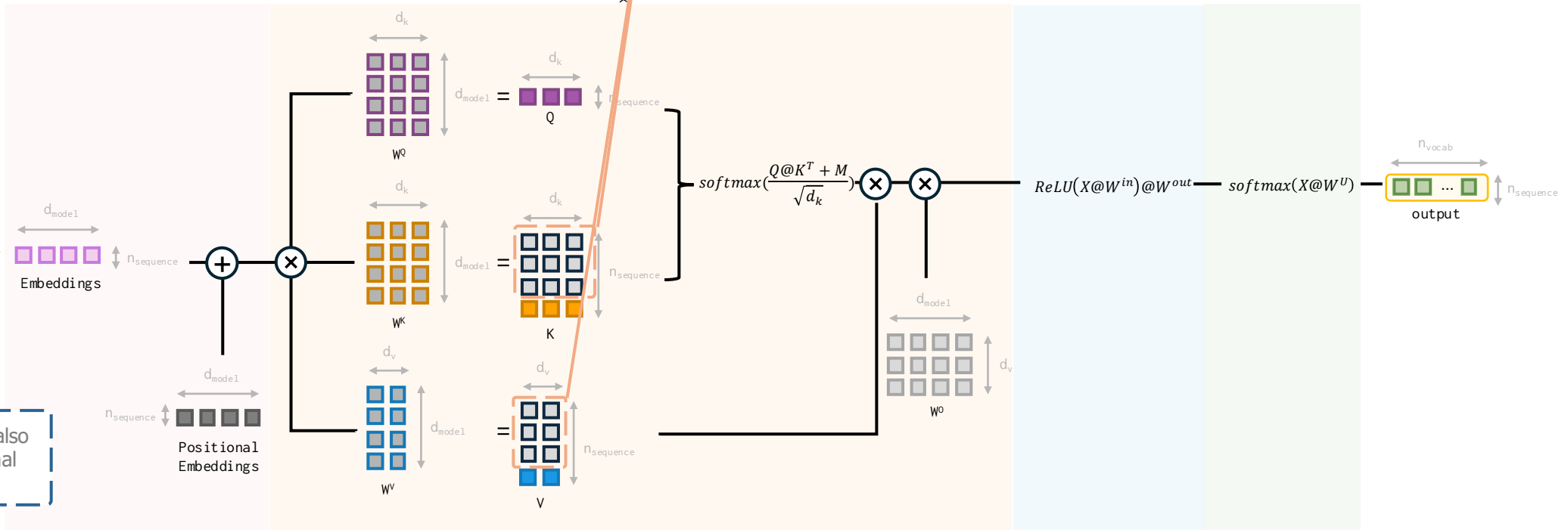
→ Cache

□ □ ... □

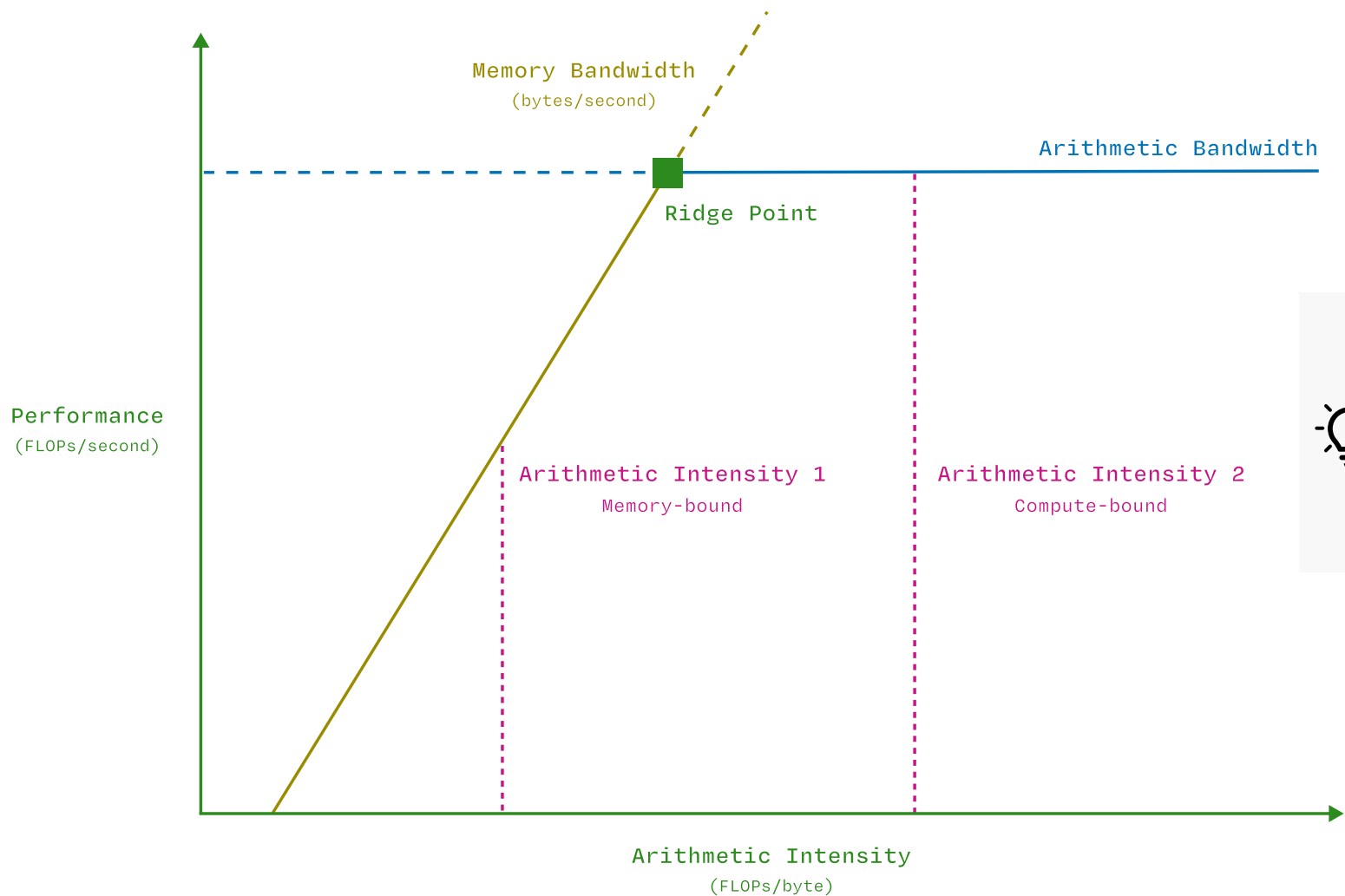
i Past tokens are only necessary in the attention mechanism. Their probability distribution are discarded.

We like dragons that
We like dragons that

2167 1299 103944 484



Note: Actual sequence length is also cached to ensure correct positional encoding



Prefilling is compute-bound

Decoding is memory bandwidth-bound

Conceptual Overview of Efficient LLM Inference for Bandwidth-Bound Tasks

Move Fewer Bytes

Read half as many bytes per step to emit twice as many tokens per second.
e.g. Quantization

Do More Work Per Byte

If data (e.g. weights) is loaded anyway, reuse it across as many operations as possible.
e.g. Batching

Improve Overlap

Reorder memory transfers and compute so neither resource sits idle.
e.g. FlashAttention



(A Bit More) Detailed Overview

Of Efficient LLM Inference

4.1

(A Bit More) Detailed Overview Of Efficient LLM Inference Data Level

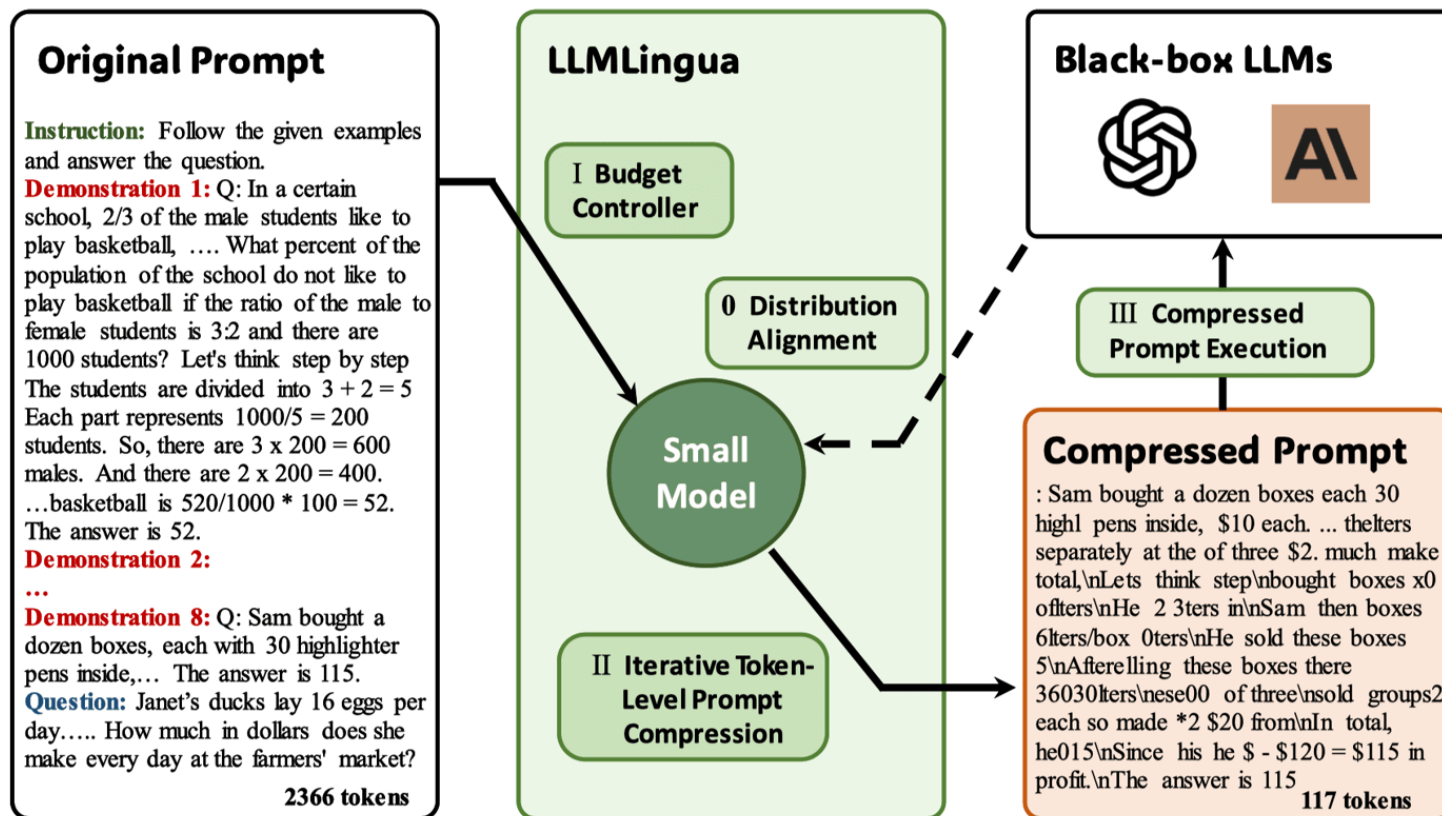
Prompt Compression

Insight

Natural Language is **redundant** & the amount of **information** of tokens varies.

Concept

Use a small language model to drop “unimportant” tokens. (e.g. through perplexity¹ threshold, BERT classification, etc.) while preserving a prompts “utility”.



<https://llmlingua.com/>

1. $1 / P(\text{token} | \text{context})$; Intuitively, it's the effective number of equally-likely guesses the model would have needed to land on the actual token: perplexity 1 means perfect prediction, 100 means the model treated the token as a 1-in-100 chance. Lower perplexity = more predictable = less informative.

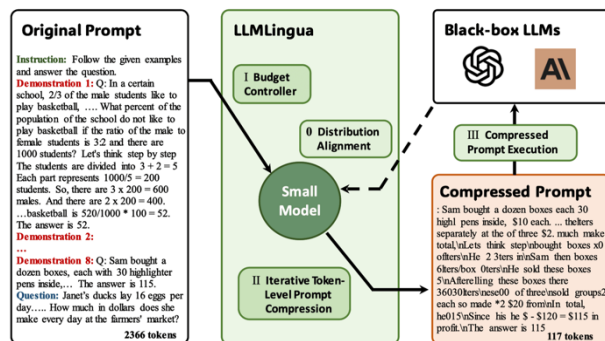
Prompt Compression

Insight

Natural Language is **redundant** & the amount of **information** of tokens varies.

Concept

Use a small language model to drop “unimportant” tokens. (e.g. through perplexity¹ threshold, BERT classification, etc.) while preserving a prompts “utility”.



<https://llmlingua.com/>

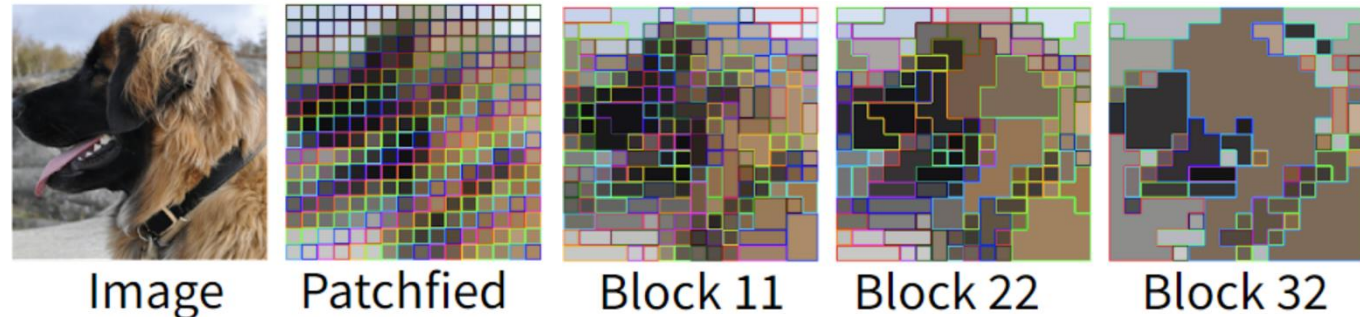
Token Merging*

Insight

Adjacent or similar KV entries often carry redundant information.

Concept

Identify similar KV pairs (e.g. by K-vector similarity, position, or a learned policy) and combine them into a single representative entry - typically by averaging K and weighted-averaging V.



[Token Merging: Your ViT But Faster](#)

At each transformer block more patches get merged together

* More prominent with Vision Transformers

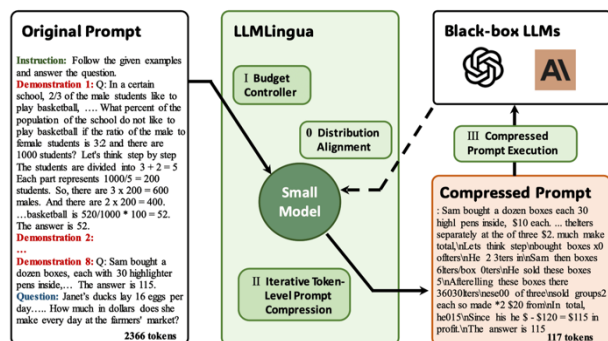
Prompt Compression

Insight

Natural Language is **redundant** & the amount of **information** of tokens varies.

Concept

Use a small language model to drop “unimportant” tokens. (e.g. through perplexity¹ threshold, BERT classification, etc.) while preserving a prompts “utility”.



<https://llmlingua.com/>

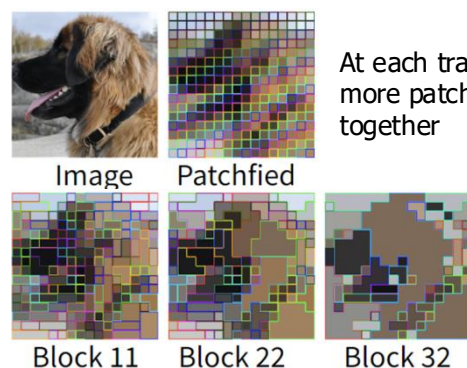
Token Merging*

Insight

Adjacent or similar KV entries often carry redundant information.

Concept

Identify similar KV pairs (e.g. by K-vector similarity, position, or a learned policy) and combine them into a single representative entry - typically by averaging K and weighted-averaging V.



[Token Merging: Your ViT But Faster](#)

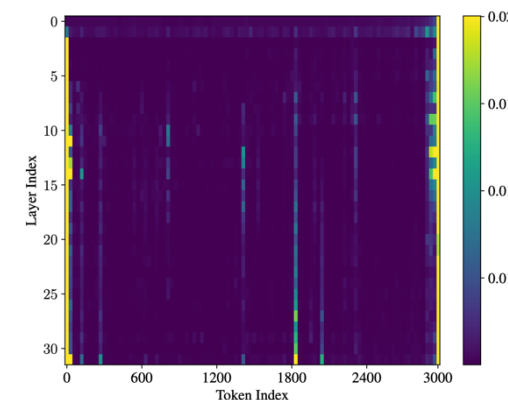
Token Pruning

Insight

Attention is sparse, Most tokens receive almost no attention from future tokens.

Concept

Score input tokens by attention received in earlier layers; tokens consistently ignored are dropped from the rest of the forward pass.



[LazyLLM: Dynamic Token Pruning for Efficient Long Context LLM Inference](#)

4.2

(A Bit More) Detailed Overview Of Efficient LLM Inference Model Level

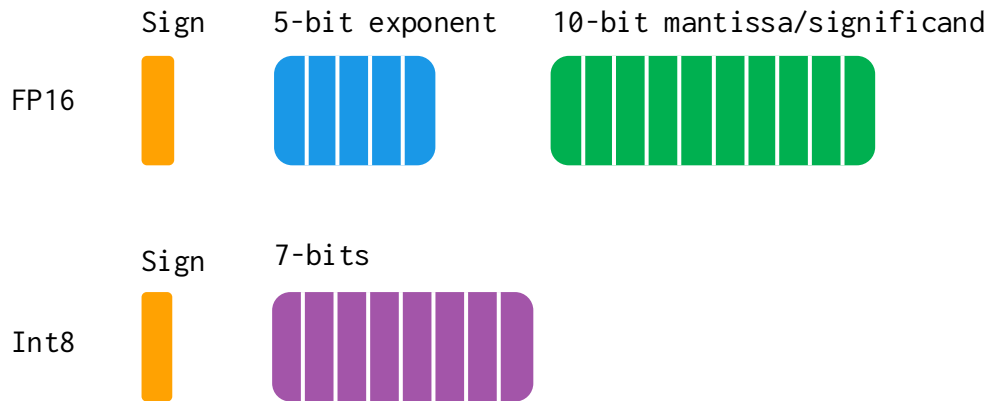


Model Level

Quantization

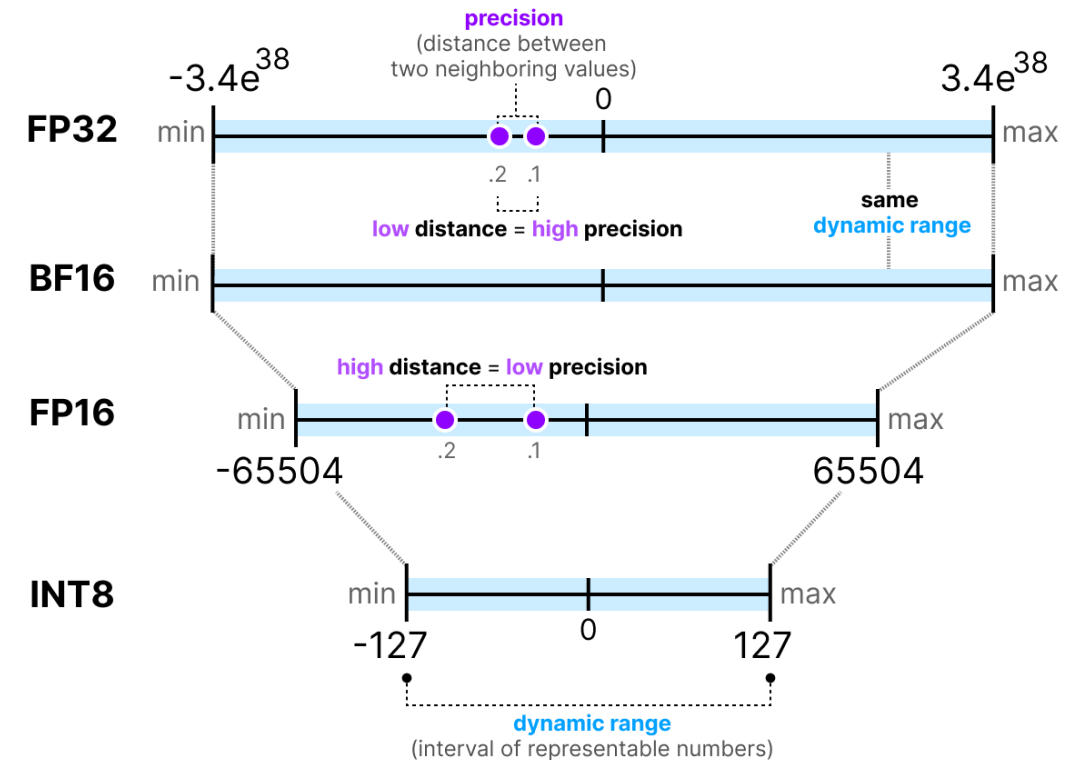


Computers store bits. Every numeric format is a convention for interpreting a bit pattern as a real number.



Why?

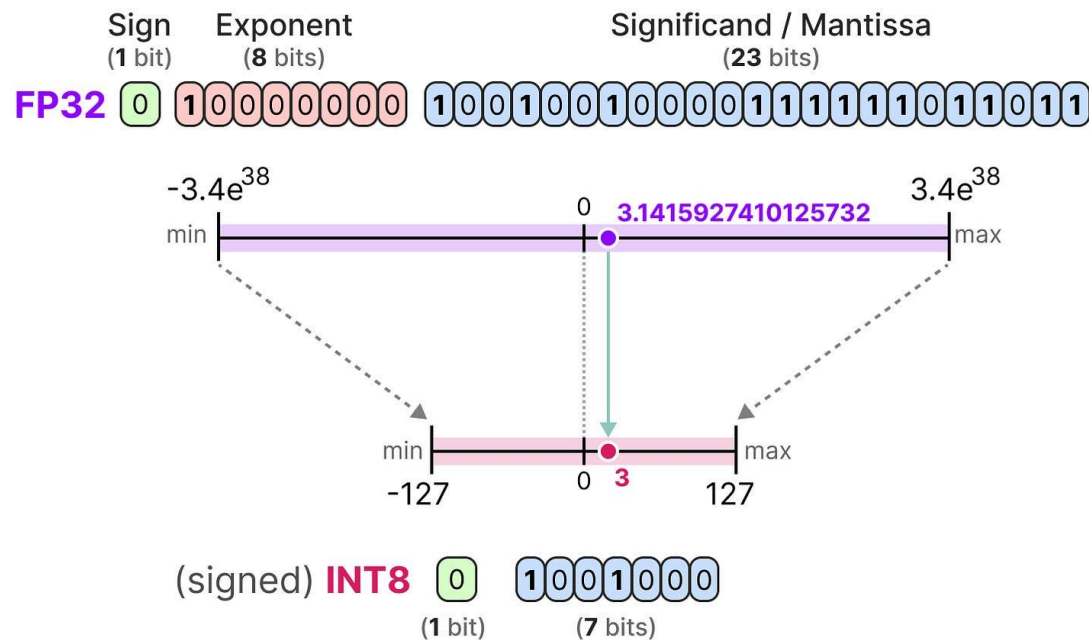
Formats are a trade-off between precision, range and memory footprint. In ML algorithms have different distributions¹ at different points in time.



<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization>

1. Range is important to preserve outliers (e.g. for training gradients). However, when outliers are handled differently (e.g. through smart scaling in quantization algorithms) precision becomes more important.

💡 Quantization **maps** values **from a high-precision** numeric format **to a lower-precision** one, trading accuracy for lower memory footprint, lower bandwidth and potentially faster compute.¹



<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization>

Naive Approach

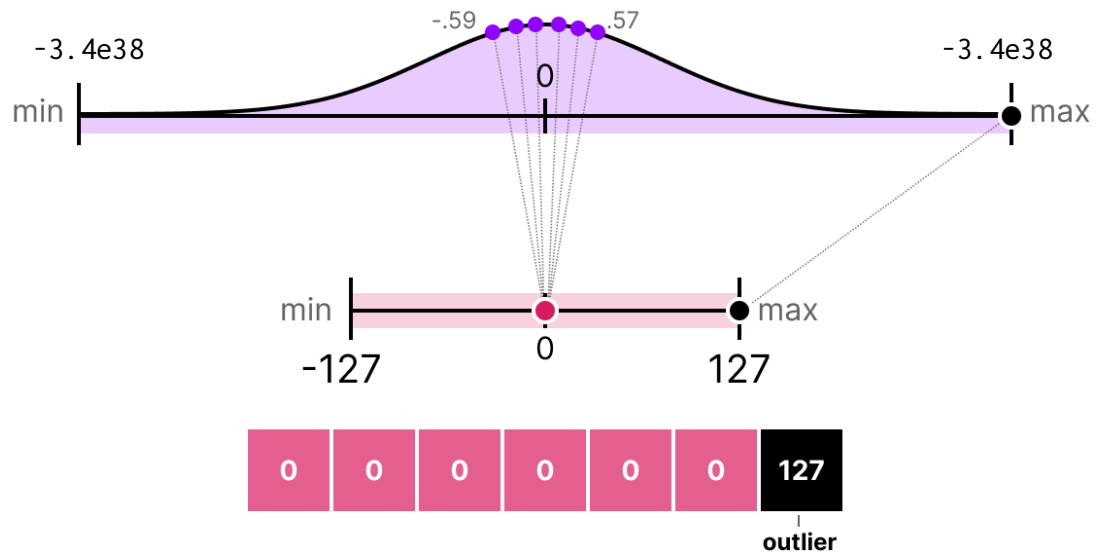
Given a symmetric² source range $[x_{\min}, x_{\max}]$ mapped to symmetric target range $[q_{\min}, q_{\max}]$:

$$\text{scale} = (x_{\max} - x_{\min}) / (q_{\max} - q_{\min})$$

$$q = \text{round}(x / \text{scale})$$

1. If also activations are quantized, computations can be done in a faster core

2. Symmetry meaning $x_{\min} = -x_{\max}$

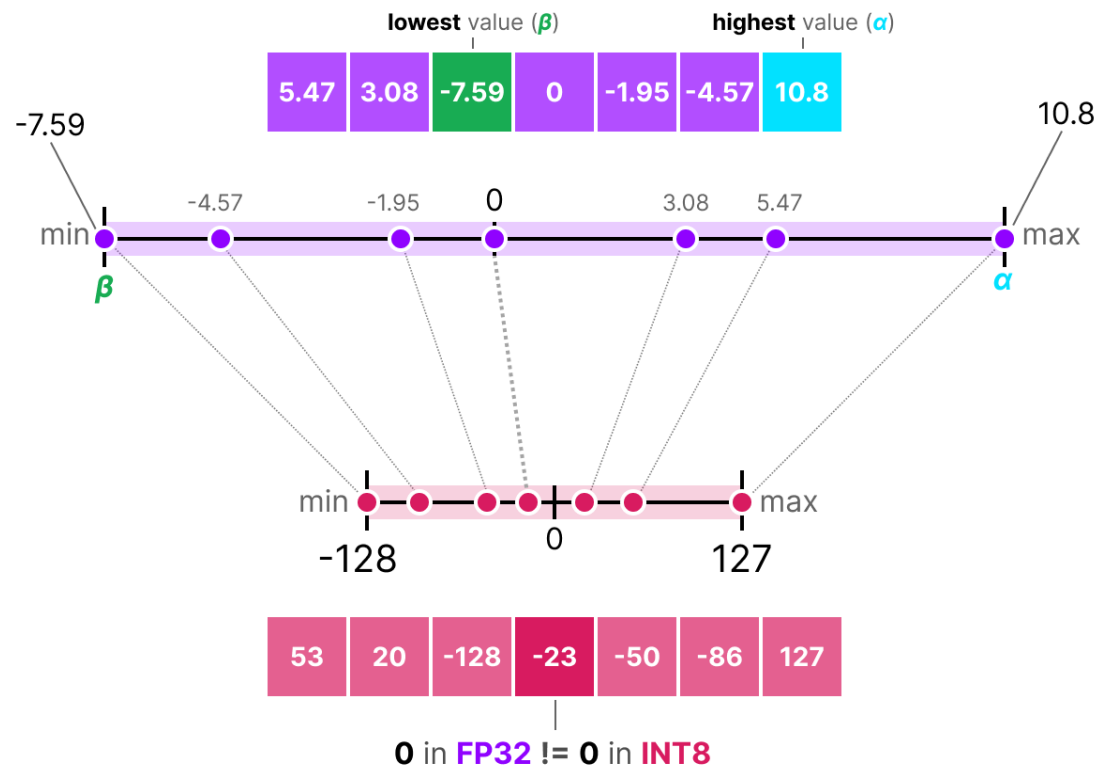


Issue

- Often weights/activations aren't distributed symmetrically
- Due to high range differences most realistic weights/activations map to 0

<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization>

Model Level: Quantization



Easy Solution

- Calculate zero-offset
- Use actual min / max values instead of theoretical format limits

<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization>

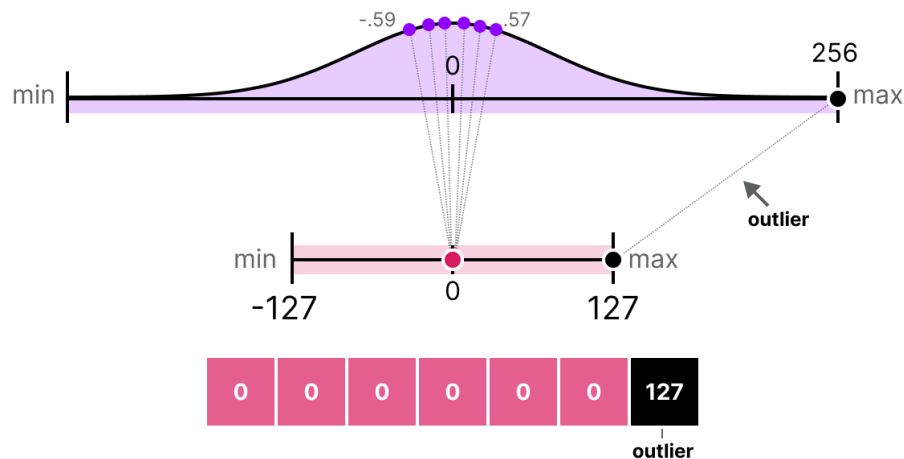
1. $z = \text{round}(q_{\min} - x_{\min} / \text{scale})$

Valid Question:

? This approach looks trivial, hardware independent and very quick to do. So why do we need costly / complex algorithms? (GPTQ, AWQ, GGUF, SmoothQuant, KIVI...)

Issue

Outliers might still scew the mapping and map most realistic weights/activations to 0

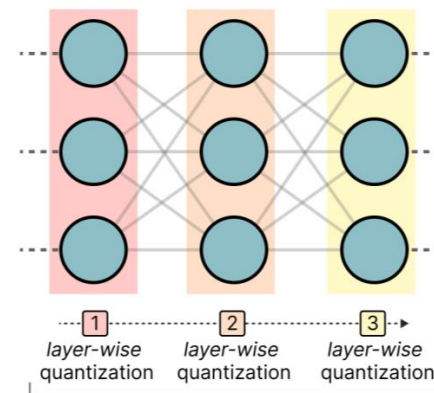


<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization>

Concept

Grouping of weights / activations preserves accuracy at low bit widths.

Dequantization¹ is fused into the kernel, which must be aware of algorithm-specific parameters and memory layout — hence different file formats and hardware dependency.



TheBloke/CapybaraHermes-2.5-Mistral-7B-GPTQ
Updated Jan 31, 2024 • ⬇ 335 • ❤ 61

TheBloke/CapybaraHermes-2.5-Mistral-7B-GGUF
7B • Updated Jan 31, 2024 • ⬇ 2.05k • ❤ 126

TheBloke/CapybaraHermes-2.5-Mistral-7B-AWQ
Updated Jan 31, 2024 • ⬇ 24 • ❤ 23

1. Dequantization is the process of mapping quantized numbers back to a higher precision format so that the kernel can handle it as kernels expect the specific number formats.

Valid Question:

? This approach looks trivial, hardware independent and very quick to do. So why do we need costly / complex algorithms? (GPTQ, AWQ, GGUF, SmoothQuant, KIVI...)

Issue

Per-group quantization minimizes how far each dequantized element is from the original.

$$\min \|W - W_Q\|_F^2$$

Goal

Optimize how far the *layer output* ends up from what it would have been in a higher precision format.

$$\min \|XW - XW_Q\|_F^2$$

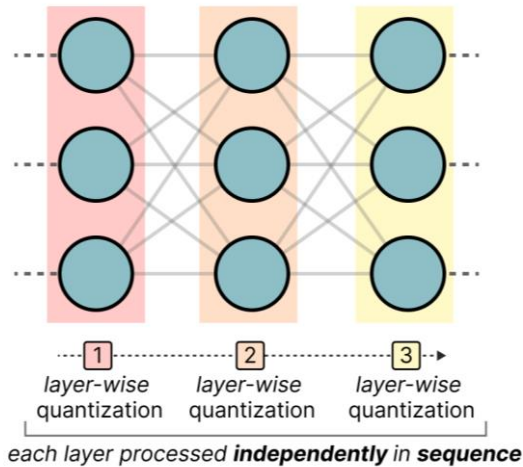
Issue

Bits spent equally produce uneven output quality, because weights aren't equal in their output impact.

Concept

"Smart" algorithms **use calibration data** to identify which **values** carry the most **signal** — salient weights get more precision.

Activation outliers are isolated or smoothed; The bit budget concentrates where errors hurt most.



<https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-quantization>

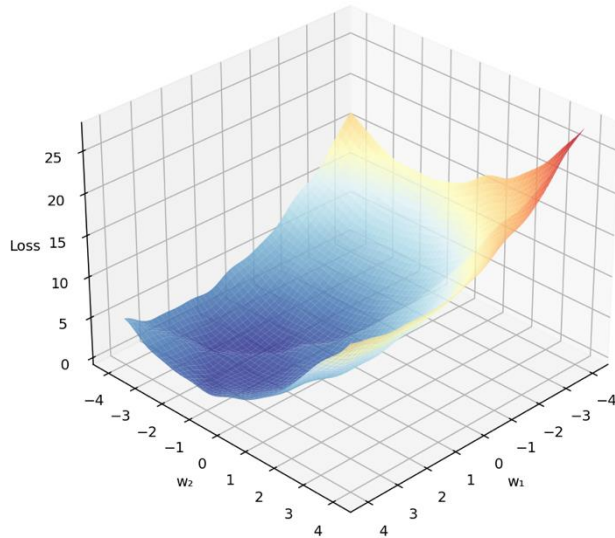
Goal is to find minimize $L = \|XW - XW_Q\|_F^2$ with $\|A\|_F = \sqrt{\sum A_{ij}^2}$
Find quantized weights W_Q such that the layer's output on calibration data is as close as possible to the original layer's output.

Important “Helper”:

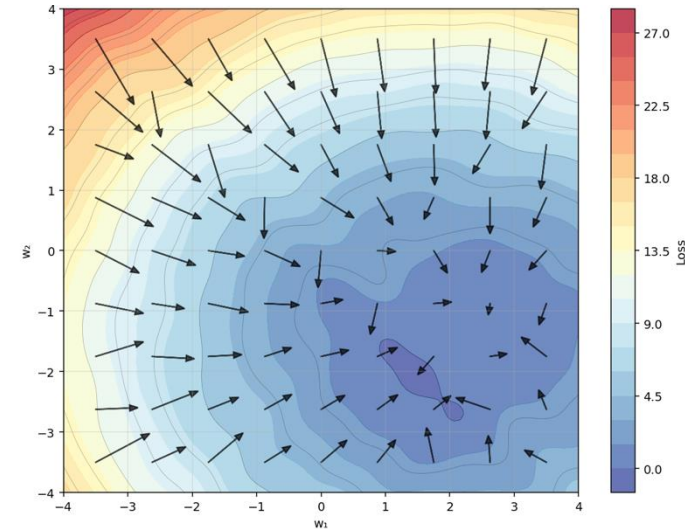
Hessian Matrix $H: \nabla^2 L(W_1, W_2) = 2X^T X$

Model Level: Quantization |GPTQ

- Goal is to find minimize $L = \|XW - XW_Q\|_F^2$ with $\|A\|_F = \sqrt{\sum A_{ij}^2}$
- Hessian Matrix $H: \nabla^2 L(W_1, W_2) = 2X^T X$



Loss Landscape $L(W_1, W_2)$



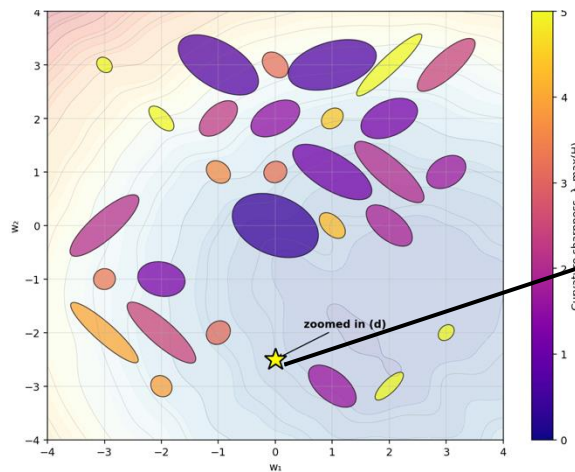
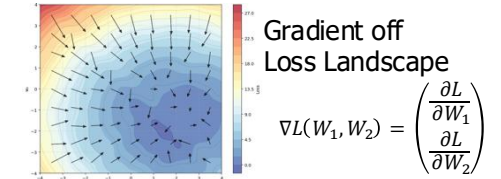
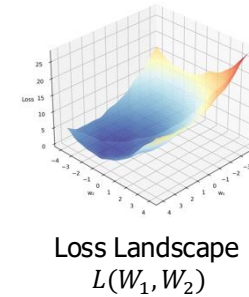
Gradient off Loss Landscape¹

$$\nabla L(W_1, W_2) = \begin{pmatrix} \frac{\partial L}{\partial W_1} \\ \frac{\partial L}{\partial W_2} \end{pmatrix}$$

1. In theory, every point would have an error. However, to improve visualisation, we only show a few arrows (i.e. a few gradient values).

Model Level: Quantization |GPTQ

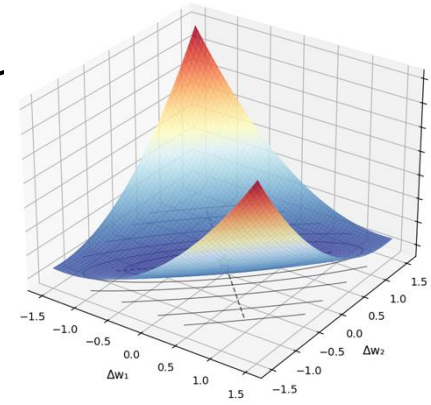
- Goal is to find minimize $L = \|XW - XW_Q\|_F^2$ with $\|A\|_F = \sqrt{\sum A_{ij}^2}$
- Hessian Matrix: $H: \nabla^2 L(W_1, W_2) = 2X^T X$



Hessian visualized as „Curvature Ellipses“

$$H: \nabla^2 L(W_1, W_2) = \begin{bmatrix} \frac{\partial^2 L}{\partial W_1^2} & \frac{\partial^2 L}{\partial W_1 \partial W_2} \\ \frac{\partial^2 L}{\partial W_2 \partial W_1} & \frac{\partial^2 L}{\partial W_2^2} \end{bmatrix}$$

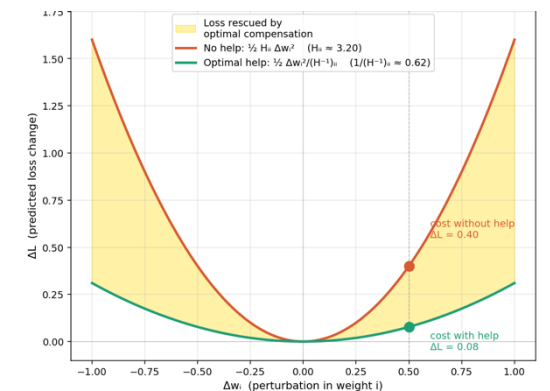
- Orientation:** The long axis of the ellipse points along the direction in which loss changes slowly. The short axis points along the direction where loss changes quickly.
- Aspect ratio:** A nearly-round ellipse means curvature is similar in all directions; a long thin cigar means one direction is dramatically sharper than the other.
- Color:** encodes $\lambda_{\max}(H)$, the largest eigenvalue¹ — the sharpness of the sharpest direction.



$$\Delta L(\Delta W_1, \Delta W_2) = \frac{1}{2} \Delta(\Delta W_1, \Delta W_2)^T H \Delta(\Delta W_1, \Delta W_2)$$

Freeze H at one location (the yellow star, at $W = (0, -2.5)$) and used it to predict ΔL for nearby perturbations.

Now you can answer, given a change in W_1 how much would I need to change W_2 to minimize the change in loss ΔL

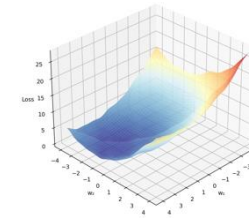


Loss sensitivity at $W = (0, -2.5)$ to changes in W_i and lowest possible loss when compensating with other weights.

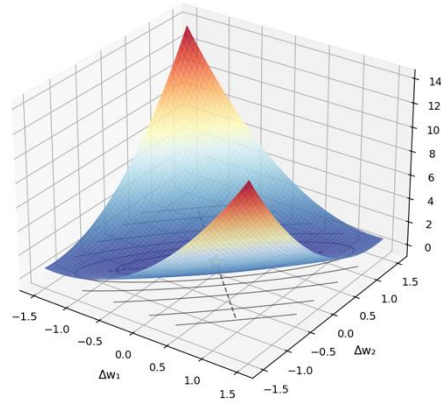
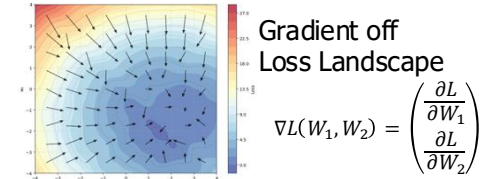
1. A matrix is a transformation (vector in rotated, stretched/sqished vector out). Eigenvectors are directions along which factors aren't rotated but only stretched. Eigenvalues are the stretching-factor. $A v = \lambda v$

Model Level: Quantization |GPTQ

- Goal is to find minimize $L = \|XW - XW_Q\|_F^2$ with $\|A\|_F = \sqrt{\sum A_{ij}^2}$
- Hessian Matrix: $H: \nabla^2 L(W_1, W_2) = 2X^T X$



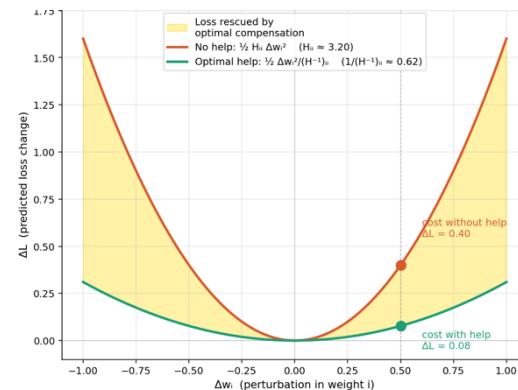
Loss Landscape
 $L(W_1, W_2)$



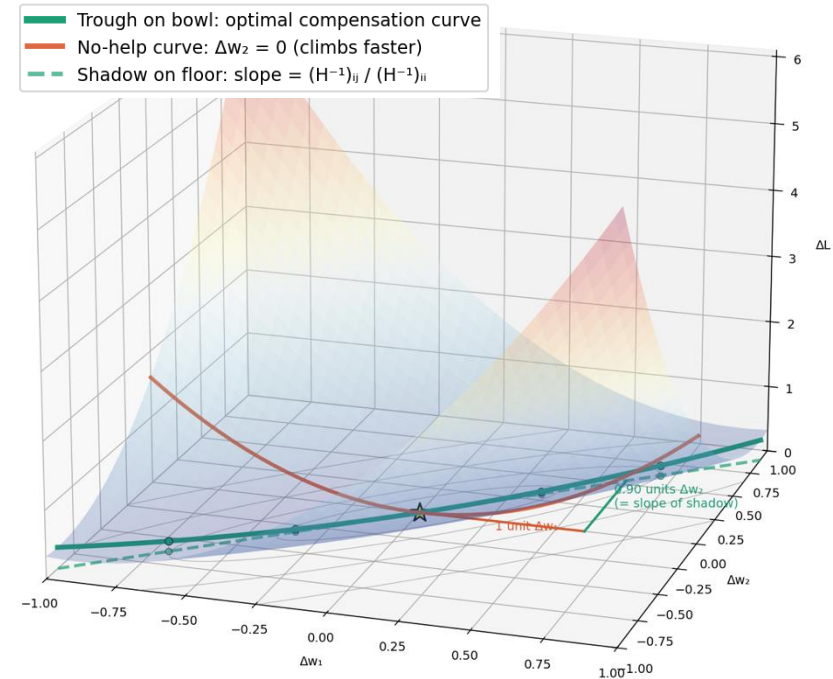
$$\Delta L(\Delta W_1, \Delta W_2) = \frac{1}{2} \Delta(\Delta W_1, \Delta W_2)^T H \Delta(\Delta W_1, \Delta W_2)$$

Freeze H at one location (the yellow star, at $W = (0, -2.5)$) and used it to predict ΔL for nearby perturbations.

Now you can answer, given a change in W_1 how much would I need to change W_2 to minimize the change in loss ΔL



Loss sensitivity at $W = (0, -2.5)$ to changes in W_i and lowest possible loss when compensating with other weights.



$\frac{H_{i,j}^{-1}}{H_{i,i}^{-1}}$ the green slope of the line on the floor showing how far weight j must move per unit of weight i 's perturbation to keep the loss minimal.

- Goal is to find minimize $L = \|XW - XW_Q\|_F^2$ with $\|A\|_F = \sqrt{\sum A_{ij}^2}$

Find quantized weights W_Q such that the layer's output on calibration data is as close as possible to the original layer's output.

- Hessian Matrix $H: \nabla^2 L(W_1, W_2) = 2X^T X$ and compute inverse H^{-1}

The Hessian tells you the *curvature* of the loss. A large H_{ii} means "weight i is very sensitive; changing it moves the loss a lot." A small H_{ii} means "weight i is unimportant; changing it has a small impact on loss." The off-diagonal¹ H_{ij} tells you how weights are coupled (i.e. if you change weight i, can weight j compensate?).

The inverse Hessian H^{-1} flips the perspective. A large $H_{i,i}^{-1}$ means "perturbing weight i is expensive — even with help from the others, the loss still goes up a lot." A small $H_{i,i}^{-1}$ means "perturbing weight i is cheap — the other weights can absorb the damage efficiently." The off-diagonal $H_{i,j>i}^{-1}$ tells you how that absorption happens — the optimal share weight j should take on when weight i is perturbed.

1. Elements not on the diagonal

- Goal is to find minimize $L = \|XW - XW_Q\|_F^2$ with $\|A\|_F = \sqrt{\sum A_{ij}^2}$
- Hessian Matrix $H: \nabla^2 L(W_1, W_2) = 2X^T X$ and compute inverse H^{-1}
- Initialize $W_Q = \text{zeros}(d_{in}, d_{out})$
- For each $w_i \in W$:
 - $q_i = \text{quant}(w_i)$ # simple map as previously discussed
 - $W_Q[i, :] = q_i$ # store quantized value in final output
 - $W[j > i, :] -= \frac{(w_i - q_i)}{H_{i,i}^{-1}} H_{i,j>i}^{-1}$ # error compensation

The rounding error $(w_i - q_i)$ is converted into a "compensation force" by dividing by $H_{i,i}^{-1}$.

That force is then distributed across the remaining weights via the off-diagonals $H_{i,j}^{-1}$, so each one absorbs the share that minimizes the overall loss.

4.2.2

Model Level

Attention-Optimization



Recap:

The scaled dot product between Query and Key lead to attention scores. The softmax normalizes them to attention weights, which multiplied by the Value create the final output.

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Or

$$S = Q@K^T$$

$$P = \text{softmax}\left(\frac{S}{\sqrt{d_k}}\right)$$

$$O = P@V$$

$$\text{with } \text{softmax} = \frac{e^{x_i - m}}{\sum_{j=1}^N e^{x_j - m}}$$

```
output = np.zeros(len(S), dtype=float)

# First pass: find max
max_val = S[0]
for i in range(1, n):
    if input_array[i] > max_val:
        max_val = input_array[i]

# Second pass: compute exp(x - max) and sum
sum_val = 0.0
for i in range(n):
    output[i] = np.exp(input_array[i] - max_val)
    sum_val += output[i]

# Third pass: normalize
for i in range(n):
    output[i] /= sum_val

print(output)
```

Issues:

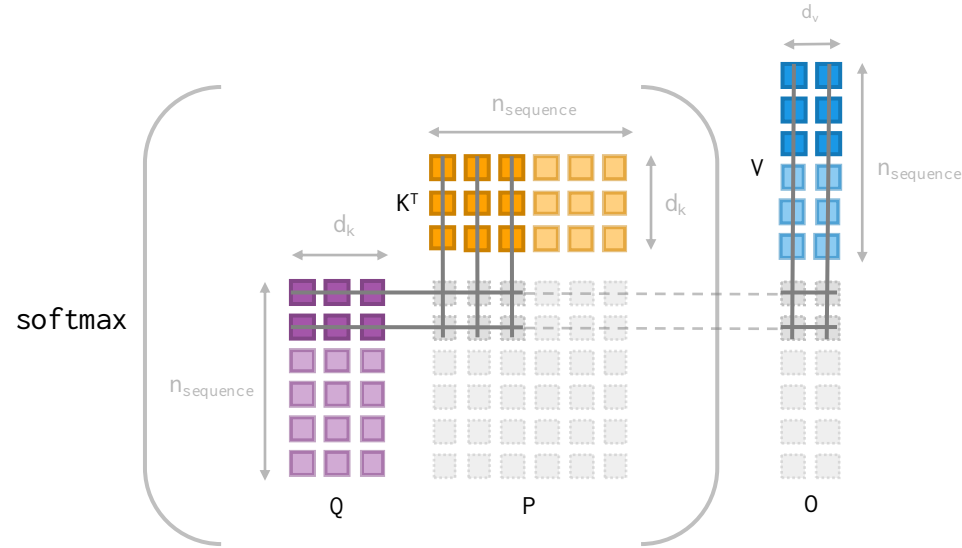
- Multiple reads from / writes to HBM
- When S does not fit in one kernel (usually >32k tokens) softmax must read each tile 3 times and write intermediate results

* softmax is shiftinvariant ($\text{softmax}(x) = \text{softmax}(x-c)$). Subtracting the rows max helps numerical stability in the exponent.



Attention-Optimization

Example FlashAttention



Benefits:

- Divide large operations into small parallelizable chunks
- Read each tile once and write result once

B_r ... Q-tile row count
B_c ... KV-tile row count

```
for Q_i in Q_tiles:
    m_i = full((B_r,), -inf)
    l_i = zeros((B_r,))
    O_i = zeros((B_r, d_k))

    for K_j, V_j in KV_tiles
        S_ij = Q_i @ K_j.T / sqrt(d_k)
        m_ij = max(S_ij)
        m_new = max(m_i, m_ij)
        P_ij = np.exp(S_ij - m_new)

        alpha = np.exp(m_i - m_new)
        l_i = alpha * l_i + P_ij.sum(axis=1)

        O_i = alpha[:, None] * O_i + P_ij @ V_j

        m_i = m_new

O_i = O_i / l_i[:, None]
```

Q_i: (B_r, d_k)
running max; one $-\text{inf}$ for each row
running sum; one 0 for each row
output matrix for tile

K_j, V_j: (B_c, d)
(B_r, B_c)
(B_r,)
(B_r,)
(B_r, B_c) ; shift invariance

write to HBM once

Rule: $\exp(s - m_{\text{new}}) = \exp(s - m_{\text{old}}) \cdot \underbrace{\exp(m_{\text{old}} - m_{\text{new}})}_{\text{alpha}}$



Attention-Optimization

Other Techniques

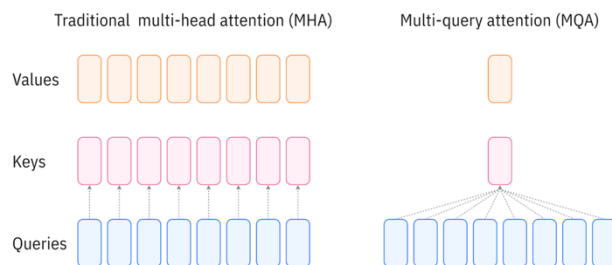
Multi-Query Attention (MQA)

Insight

Per-head K and V largely encode redundant information across heads.

Concept

Keep h query heads but share one K and one V . KV cache shrinks by h times. Big bandwidth win, but a noticeable quality drop vs. MHA.



[Fast Transformer Decoding: One Write-Head is All You Need](#)

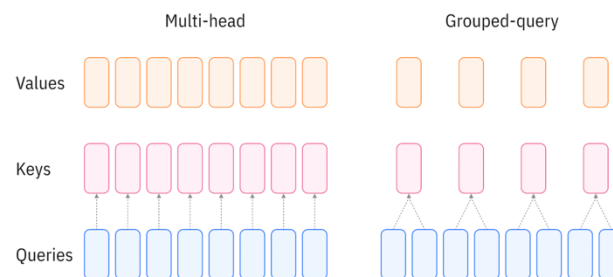
Grouped Query Attention (GQA)

Insight

MQA's full sharing is too aggressive — but per-head sharing (MHA) is overkill.

Concept

Partition the h query heads into g groups, each sharing one K and one V . Cache shrinks by h/g times. Recovers near-MHA quality at near-MQA cost.



[GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints](#)

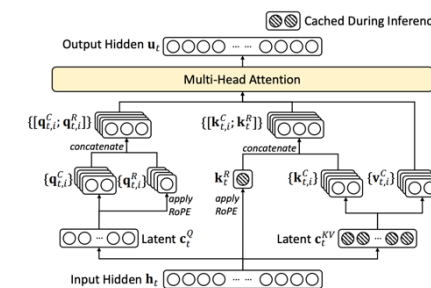
Multi-head Latent Attention (MLA)

Insight

Per-head K and V largely encode redundant information across heads.

Concept

Project tokens to a shared latent c and cache only c . Decompress to per-head K and V at attention time via learned projections that fold into Q 's matrix. Around 10–20 times smaller cache than MHA, with often better quality.



[DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model](#)

Paged Attention

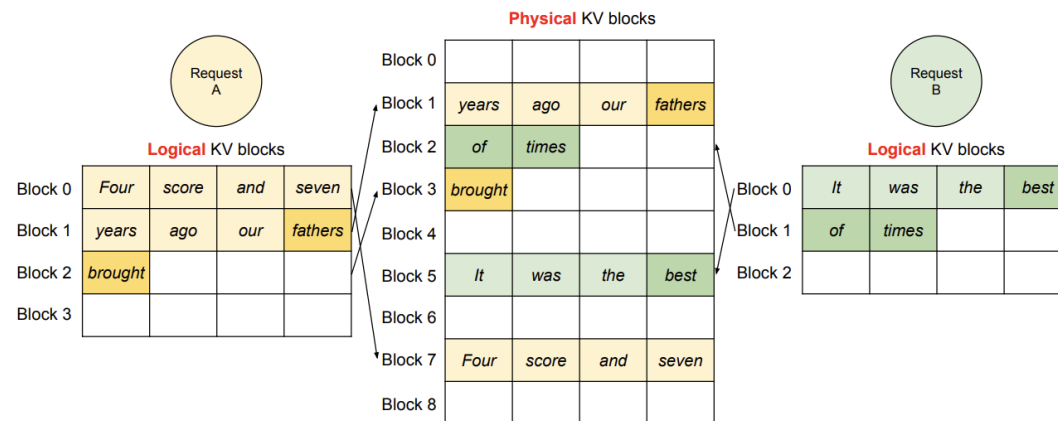
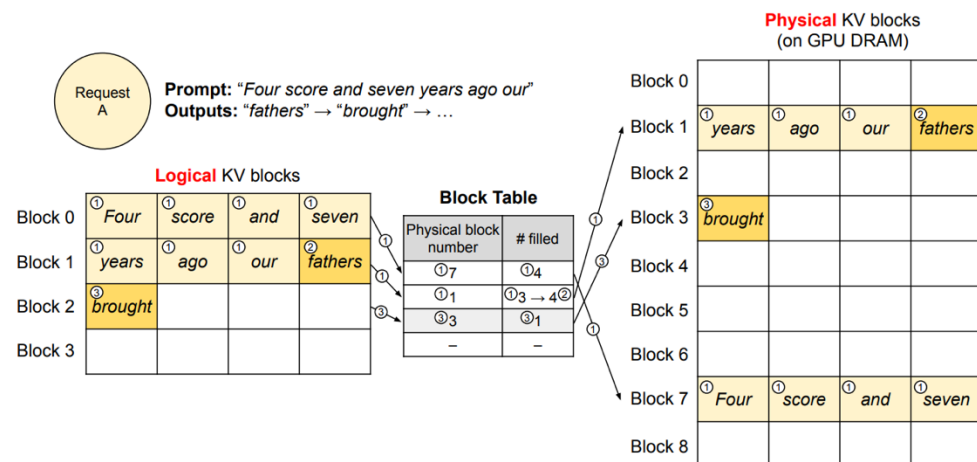
- Naive KV cache allocation:**
Reserve max_sequence length in KV cache for each decoding request → Memory Capacity Issue
- Reserve only what you need:**
As requests start/finish at different times KV cache becomes fragmented → Natural fragmentation that becomes worse / “unstable” across multiple requests* and introduces additional overhead

- Cache Duplication:**
Each request stores complete KV cache (e.g. system-prompt is fully duplicated)

Solution:

Paged Attention: fragmented on purpose, with fixed block size

[Efficient Memory Management for Large Language Model Serving with Paged Attention](#)



* Meaning it leads to randomly sized memory slots

4.2.3

Model Level

Smart Decoding

Is there any way to generate more than one token per forward pass?

No

However, *verifying* a hypothesised sequence of tokens is cheaper than *generating* them

Speculative Decoding

Insight

Verifying k tokens in parallel costs almost the same as generating one.

Concept

A small draft model autoregressively proposes k tokens. The target model verifies all k positions in a single forward pass. On the first rejection, fall back and resample.

[Fast Inference from Transformers via Speculative Decoding](#)

WITHOUT SPECULATIVE DECODING



My favorite thing about fall

WITH SPECULATIVE DECODING



My favorite thing about fall

Speculative Decoding

Insight

Verifying k tokens in parallel costs almost the same as generating one.

Concept

A small draft model autoregressively proposes k tokens. The target model verifies all k positions in a single forward pass. On the first rejection, fall back and resample.

WITHOUT SPECULATIVE DECODING



My favorite thing about fall is the

WITH SPECULATIVE DECODING



My favorite thing about fall is the change in color

[Fast Inference from Transformers via Speculative Decoding](#)

Speculative Decoding Without Draft Model

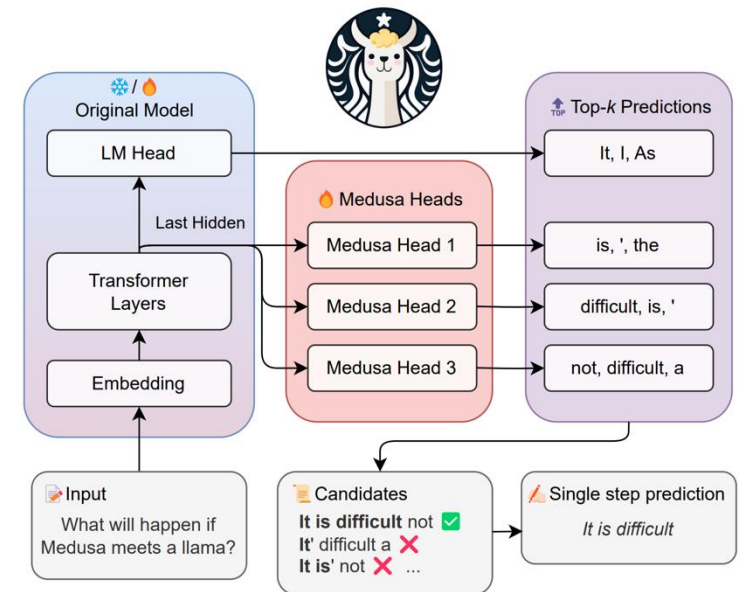
Insight

A separate draft model is overhead complexity.

Concept

Add k lightweight prediction heads on top of the target model itself, each trained to predict positions $+1, +2, \dots, +k$ from the same hidden state. One forward pass yields k speculation candidates. Tree attention verifies multiple candidate continuations simultaneously. ^

[Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads](#)



1. It's basically just slightly more advanced masking that ensures each predicted token only refers to the original text and the preceding tokens from the same head, rather than the siblings.

Speculative Decoding

Insight

Verifying k tokens in parallel costs almost the same as generating one.

Concept

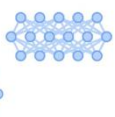
A small draft model autoregressively proposes k tokens. The target model verifies all k positions in a single forward pass. On the first rejection, fall back and resample.

WITHOUT SPECULATIVE DECODING



My favorite thing about fall is the

WITH SPECULATIVE DECODING



My favorite thing about fall is the change in color

[Fast Inference from Transformers via Speculative Decoding](#)

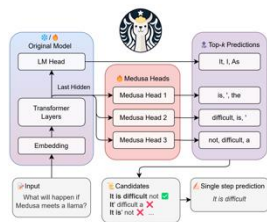
Speculative Decoding Without Draft Model

Insight

A separate draft model is overhead complexity.

Concept

Add k lightweight prediction heads on top of the target model itself, each trained to predict positions $+1, +2, \dots, +k$ from the same hidden state. One forward pass yields k speculation candidates. Tree attention verifies multiple candidate continuations simultaneously.



[Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads](#)

Multi Token Prediction (MTP)

Insight

Standard training just optimizes for the one-step-ahead view (i.e. drafts bolted on afterward (Medusa) are limited).

Concept

During pretraining, attach k sequential MTP modules, each predicting positions $+1, +2, \dots, +k$.

Module k conditions on module $k-1$'s hidden state — a natively autoregressive draft chain instead of parallel heads. At inference, the same chain serves as a built-in speculator: high acceptance, no auxiliary model. Used in DeepSeek-V3.

[Better & Faster Large Language Models via Multi-token Prediction](#)

4.2.4

Model Level

Conditional Compute

Width-wise (Mixture-of-Experts)

Insight

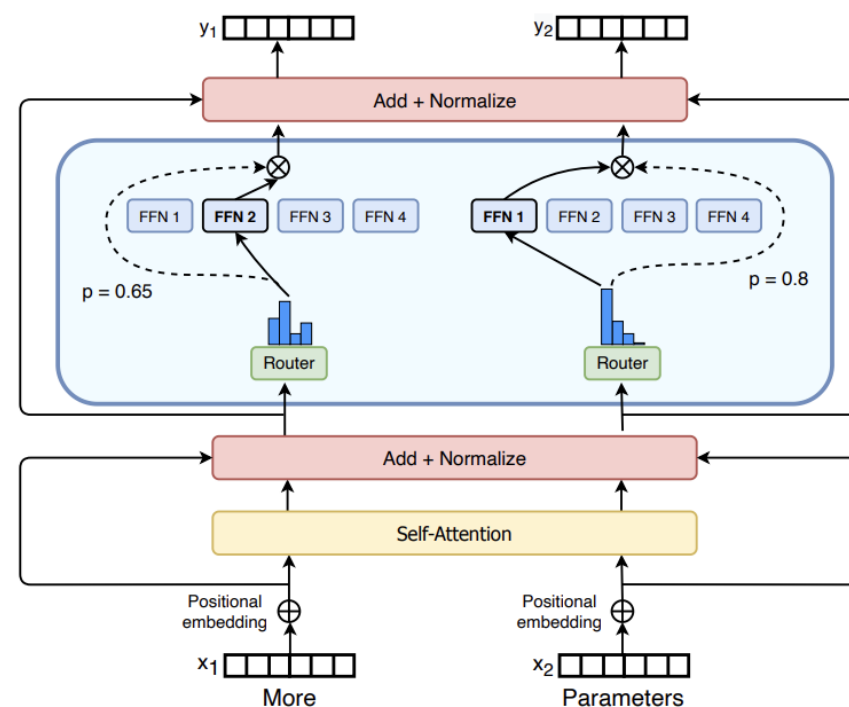
Different tokens need different specialized circuits¹ – running every FFN² parameter on every token is wasteful.

Concept

Replace dense FFN with N parallel small networks (experts). A router picks the top-k experts per token; only those experts run. Total parameters scale with N, per-token compute stays roughly constant; bigger model capacity at fixed inference cost.

Router is typically a simple Linear layer with softmax on top. Even distribution in training is ensured with auxiliary load-balancing, routing bias and/or capacity mechanisms.

[Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity](#)



1. A small specialized assembly of weights/neurons that collectively compute one specific function
2. Feed Forward Network or Multi Layer Perceptron (MLP)

Width-wise (Mixture-of-Experts)

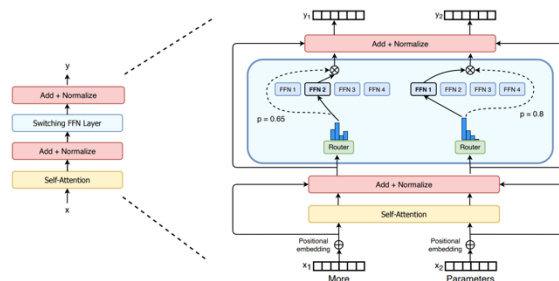
Insight

Different tokens need different specialized circuits¹ – running every FFN² parameter on every token is wasteful.

Concept

Replace dense FFN with N parallel small networks (experts). A router picks the top-k experts per token; only those experts run. Total parameters scale with N, per-token compute stays roughly constant; bigger model capacity at fixed inference cost.

Router is typically a simple Linear layer with softmax on top. Even distribution in training is ensured with auxiliary load-balancing or routing bias.



[Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity](#)

Depth-wise (Early Exit, Mixture-of-Depths, LayerSkip, ...)

Insight

Not all tokens are equally "hard" for the model (e.g. "comma" vs. complex reasoning steps).

Concept

Add per-token routing to the depth dimension: each token can skip layers it doesn't need, riding the residual connection instead.

- [Mixture-of-Depths \(MoD\)](#): learned router per layer, top-k tokens enter, deterministic compute.
- [Early Exit](#): confidence threshold per layer, token leaves the stack once sure.
- [LayerSkip](#): trained with random layer dropout, partial-depth runs reused as speculation drafts.

1. a small specialized assembly of weights/neurons that collectively compute one specific function
2. Feed Forward Network or Multi Layer Perceptron (MLP)

4

Recap / Closing Remarks

Where to go from now?

One Slide Summary

- LLM inference is mostly memory-bandwidth bound, not compute bound. Tensor cores have FLOPs to spare; HBM bandwidth is the bottleneck. Most optimizations are some variation of "load fewer bytes from HBM" or "reuse what's loaded longer" or "hide costly loading processes behind other processes".
- Prefill and decode are different phases. Prefill does big matmuls (compute-bound, batch-friendly). Decode does single-token forward passes (memory-bandwidth bound, dominated by weight + KV cache loads). Optimizations target different bottlenecks in each.
- The KV-cache-related tasks often dominate. Strategies shrink, compress, tile and/or reuse it.
- fused kernel and online softmax.
- Conditionally compute saves work, not memory. MoE skips experts (width); MoD skips layers (depth)
- Quantization uses calibration-aware methods (GPTQ, AWQ) to minimise output error over-weight-error. Each format ships paired with custom dequantization kernels.
- Speculative decoding gets free speedup with no quality loss. A draft proposes, the target verifies in parallel

Where to go from here

- [vLLM](#) documentation for implementation details and [blog](#) for deep-dives
- What we skipped:
- Multi-GPU serving: tensor parallelism, pipeline parallelism, expert parallelism. Megatron-LM and Alpa are the references.
- Hardware Specifics: TPUs (Google), MI300 (AMD), LPU (Groq), Trainium (AWS), Apple Silicon.
- System-level optimization: continuous batching, request scheduling, disaggregated prefill/decode (DistServe). vLLM and SGLang docs cover the practical side.